Parte 5

Form submit com fetch: loading, erros, sucesso e integração com estado

0.1. Nesta parte

Contents

0.1. Nesta parte 2
1. Capítulo 13 — DOM e UI: formulários, estado e arquitetura de interação 3
1.1. Parte 5 — Form submit com fetch: loading, erros, sucesso e integração com estado 3
1.2. 1) Requisitos do exemplo 3
1.3. 2) index.html (mínimo) 3
1.4. 3) app.js (state/actions/render + fetch robusto) 4
1.5. 4) O porquê das decisões 8
1.6. 5) Checklist profissional (Parte 5) 8

1. Capítulo 13 — DOM e UI: formulários, estado e arquitetura de interação

1.1. Parte 5 — Form submit com fetch: loading, erros, sucesso e integração com estado

Agora você vai juntar as peças: formulário + validação + estado + fetch. O objetivo é criar um submit profissional: quando o usuário envia, você valida localmente, monta payload, entra em loading, faz request com timeout/cancelamento, trata status e erros, e atualiza a UI conforme o resultado. O ponto júnior é previsibilidade: um único estado controla o que aparece na tela (botões desabilitados, spinner, mensagens, resultado).

1.2. 1) Requisitos do exemplo

- Form com campos: nome, email, idade.
- Validação por campo com mensagens (cap. 13 parte 3).
- Estado: idle/loading/success/error + erros por campo.
- Submit: fetch POST JSON com timeout (AbortController).
- Desabilitar botão e mostrar spinner durante loading.
- Renderizar resultado em success; mensagem em error.
- Evitar duplo submit: se já estiver loading, ignore ou cancele anterior.

1.3. 2) index.html (mínimo)

```html
<! doctype html >
< html lang = "pt-BR" >
  < head >
    < meta charset = "utf-8"  />
    < meta name = "viewport"  content = "width=device-width, initial-scale=1"  />
    < title > Submit — Cap.  13 </ title >
    < script src = "app.js"  defer ></ script >
  </ head >
  < body >
    < h1 > Cadastro </ h1 >

    < form id = "form" >
      < label >
       Nome
        < input id = "nome"  />
        < small data - err = "nome" ></ small >
      </ label >

      < label >
       Email
        < input id = "email"  />
        < small data - err = "email" ></ small >
      </ label >

      < label >
```

```
26          Idade
27            < input id = "idade"  inputmode = "numeric"   />
28            < small data - err = "idade" ></ small >
29          </ label >
30
31          < button id = "submit"  type = "submit" > Enviar </ button >
32          < button id = "cancel"  type = "button" > Cancelar </ button >
33
34          < p id = "spinner"  style = "display:none" > ⏳ enviando... </ p >
35          < p id = "msg" ></ p >
36          < pre id = "out" > — </ pre >
37        </ form >
38      </ body >
39    </ html >
```

1.4. 3) app.js (state/actions/render + fetch robusto)

```javascript
1  'use strict' ;
2
3  const  ENDPOINT  =  'https://jsonplaceholder.typicode.com/posts' ;  // troca para sua API
4
5  function  assertEl (sel) {
6    const  el  =  document. querySelector (sel);
7    if  ( ! el)  throw  new  Error ( `Elemento não encontrado: ${sel}` );
8    return  el;
9  }
10
11  const  form  =  assertEl ( '#form' );
12  const  elNome  =  assertEl ( '#nome' );
13  const  elEmail  =  assertEl ( '#email' );
14  const  elIdade  =  assertEl ( '#idade' );
15  const  btnSubmit  =  assertEl ( '#submit' );
16  const  btnCancel  =  assertEl ( '#cancel' );
17  const  elSpinner  =  assertEl ( '#spinner' );
18  const  elMsg  =  assertEl ( '#msg' );
19  const  elOut  =  assertEl ( '#out' );
20
21  // ———————————————-
22  // Estado
23  // ———————————————-
24  let  state  =  {
25    status :  'idle' ,        // 'idle' | 'loading' | 'success' | 'error'
26    fieldErrors :  {},
27    message :  '' ,
28    result :  null ,
29  };
30
31  let  currentController  =  null ;
32
33  // ———————————————-
34  // Normalização/Validação
35  // ———————————————-
```

```
36 function normText (x) {
37   return String (x ?? '' ). trim (). replace ( / \s +/ g, ' ' );
38 }
39
40 function toNumberOrNull (raw) {
41   const s = String (raw ?? '' ). trim (). replace ( ',' , '.' );
42   if (s.length === 0 ) return null ;
43   const n = Number (s);
44   return Number. isNaN (n) ? null : n;
45 }
46
47 function validarNome (raw) {
48   const v = normText (raw);
49   if (v.length < 3 ) return { ok : false , msg : 'nome deve ter 3+ caracteres' };
50   return { ok : true , value : v };
51 }
52
53 function validarEmailSimples (raw) {
54   const email = normText (raw). toLowerCase ();
55   if ( ! email) return { ok : false , msg : 'email obrigatório' };
56   const parts = email. split ( '@' );
57   if (parts.length !== 2 ) return { ok : false , msg : 'email inválido' };
58   const [u, d] = parts;
59   if ( ! u || ! d || ! d. includes ( '.' )) return { ok : false , msg : 'email inválido' };
60   return { ok : true , value : email };
61 }
62
63 function validarIdade (raw) {
64   const n = toNumberOrNull (raw);
65   if (n === null ) return { ok : false , msg : 'idade inválida' };
66   if ( ! Number. isInteger (n)) return { ok : false , msg : 'idade deve ser inteira' };
67   if (n < 0 || n > 130 ) return { ok : false , msg : 'idade fora do intervalo' };
68   return { ok : true , value : n };
69 }
70
71 function validarForm () {
72   const erros = {};
73
74   const n = validarNome (elNome.value);
75   if ( ! n.ok) erros.nome = n.msg;
76
77   const e = validarEmailSimples (elEmail.value);
78   if ( ! e.ok) erros.email = e.msg;
79
80   const i = validarIdade (elIdade.value);
81   if ( ! i.ok) erros.idade = i.msg;
82
83   const ok = Object. keys (erros).length === 0 ;
84   if ( ! ok) return { ok : false , erros };
85
86   return { ok : true , payload : { nome : n.value, email : e.value, idade : i.value } };
87 }
88
89 // ———————————-
90 // Render
91 // ———————————-
```

```
92  function limparErros () {
93    for ( const el of document. querySelectorAll ( '[data-err]' )) el.textContent = '' ;
94  }
95
96  function renderErros (erros) {
97    limparErros ();
98    for ( const [campo, msg] of Object. entries (erros)) {
99      const el = document. querySelector ( `[data-err="${campo}"]` );
100       if (el) el.textContent = msg;
101   }
102 }
103
104 function render () {
105   btnSubmit.disabled = state.status === 'loading' ;
106   elSpinner.style.display = state.status === 'loading' ? 'block' : 'none' ;
107
108    renderErros (state.fieldErrors);
109
110   elMsg.textContent = state.message;
111   elOut.textContent = state.result ? JSON. stringify (state.result, null , 2 ) : '—' ;
112 }
113
114 function setState (patch) {
115   state = { ...state, ...patch };
116    render ();
117 }
118
119 // ———————————————-
120 // Rede (timeout/cancelamento + parse seguro)
121 // ———————————————-
122 async function fetchWithTimeout (url, options = {}, ms = 4000 ) {
123    const controller = new AbortController ();
124    const id = setTimeout (() => controller. abort (), ms);
125
126    try {
127      return await fetch (url, { ...options, signal : controller.signal });
128   } finally {
129      clearTimeout (id);
130   }
131 }
132
133 async function safeReadJson (res) {
134    const text = await res. text (). catch (() => '' );
135    try {
136      return { ok : true , value : JSON. parse (text) };
137   } catch {
138      return { ok : false , msg : 'JSON inválido' , raw : text };
139   }
140 }
141
142 // ———————————————-
143 // Actions
144 // ———————————————-
145 async function submit () {
146    if (state.status === 'loading' ) return ; // evita duplo submit
147
```

```
148     const v = validarForm ();
149     if ( ! v.ok) {
150       return setState ({ status : 'idle' , fieldErrors : v.erros, message : 'Corrija os campos' , result :
null  });
151     }
152
153     // limpa erros e inicia loading
154     setState ({ status : 'loading' , fieldErrors : {}, message : '' , result : null  });
155
156     // cancela request anterior (se existir)
157     if (currentController) currentController. abort ();
158    currentController = new AbortController ();
159
160     let res;
161     try {
162      res = await fetchWithTimeout (ENDPOINT, {
163        method : 'POST' ,
164        headers : {
165          Accept : 'application/json' ,
166           'Content-Type' : 'application/json' ,
167        },
168        body : JSON. stringify (v.payload),
169          // signal aqui seria do controller; como fetchWithTimeout cria o seu, em produção você unificaria.
170      }, 4000 );
171    } catch (e) {
172       setState ({ status : 'error' , message : (e && e.name === 'AbortError' ) ? 'timeout/
cancelado' : 'erro de rede' , result : null  });
173       return ;
174    }
175
176     const parsed = await safeReadJson (res);
177
178     if ( ! res.ok) {
179       const msg = parsed.ok ? (parsed.value ? .error ? .message ?? res.statusText) : (parsed.raw ||
res.statusText);
180       setState ({ status : 'error' , message : `HTTP ${res.status}: ${msg}` , result : null  });
181       return ;
182    }
183
184     if ( ! parsed.ok) {
185       setState ({ status : 'error' , message : 'JSON inválido' , result : null  });
186       return ;
187    }
188
189     setState ({ status : 'success' , message : 'Enviado com sucesso' , result : parsed.value });
190  }
191
192  function cancel () {
193     if (currentController) currentController. abort ();
194     setState ({ status : 'idle' , fieldErrors : {}, message : 'cancelado' , result : null  });
195  }
196
197  // ——————————————-
198  // Wire
199  // ——————————————-
200  form. addEventListener ( 'submit' , (ev) => {
```

```
  ev. preventDefault ();
   submit ();
});

btnCancel. addEventListener ( 'click' , cancel);

render ();
console. log ( '[submit] pronto' );
```

1.5. 4) O porquê das decisões

- State controla UI: loading desabilita submit e mostra spinner.
- Validação local melhora UX e reduz 400 desnecessário.
- Parse seguro evita quebrar em responses não JSON.
- Tratamento de status evita confundir 4xx/5xx com erro de rede.
- Arquitetura state/actions/render reduz bugs ao crescer.
- Cancelamento prepara o padrão para navegação e múltiplos submits.

1.6. 5) Checklist profissional (Parte 5)

- Eu valido e normalizo antes de enviar e mostro erro por campo.
- Eu controlo loading/success/error via estado central.
- Eu trato res.ok e parse seguro de JSON.
- Eu aplico timeout/cancelamento quando necessário.
- Eu evito duplo submit e estados fora de ordem.

Na Parte 6, você vai montar um mini-projeto completo: CRUD simples (lista + formulário) com estado, validação, fetch, cancelamento e render previsível.